

Python Programming

String Methods & Manipulation

Comprehensive Class Notes • Week 4

Part 1	Theory — What is a String & Immutability
Part 2	String Cleaning Methods
Part 3	Searching & Replacing
Part 4	Parsing Techniques — split() & join()

1 Theory — Understanding Strings

1.1 What is a String?

A **string** is a sequence of characters enclosed within single quotes ('...') or double quotes ("..."). In Python, strings are one of the most fundamental data types and are used to store and manipulate text-based data — names, addresses, email IDs, sentences, and more.

Real-world analogy: Think of a string like a necklace of beads, where each bead is a single character. The necklace as a whole is the string, and each bead has a fixed position (index) starting from 0.

```
# String examples
name  = "Tanishq Tyagi"
course = 'Python Programming'
digit = "42"           # still a string, not an integer!

# Accessing individual characters (indexing)
print(name[0])         # Output: T
print(name[-1])        # Output: i (last character)

# Getting the length
print(len(name))       # Output: 13

# Slicing
print(name[0:7])       # Output: Tanishq
```

Python

1.2 String Immutability

Strings in Python are **immutable** — this means once a string is created, its characters **cannot be changed** in place. Any operation that appears to modify a string actually creates a *brand new string* in memory.

```
name = "Tanishq Tyagi"

# This will cause a TypeError:
# name[0] = "k"           # ■ TypeError: str does not support item assignment

# Correct way — create a new string:
name = "k" + name[1:]     # ■ Creates a new string
print(name)              # Output: kanishq Tyagi
```

Python

Why does immutability matter? It makes strings safe to share across parts of a program without worrying that another piece of code will accidentally change your data. It also allows Python to optimise memory usage.

2 String Cleaning Methods

When working with real-world data — user inputs, form fields, CSV files — strings often contain unwanted spaces or inconsistent casing. **Cleaning methods** help us normalise this data before processing it.

Method	Description
<code>strip()</code>	Removes whitespace from both left and right sides
<code>lstrip()</code>	Removes whitespace from the left side only
<code>rstrip()</code>	Removes whitespace from the right side only
<code>lower()</code>	Converts every character to lowercase
<code>upper()</code>	Converts every character to uppercase
<code>title()</code>	Capitalises the first letter of every word

2.1 – `strip()` • `lstrip()` • `rstrip()`

These three methods deal with removing unwanted whitespace. They are essential when processing user input because users often accidentally add extra spaces.

```
name = "    JOHN DOE    "

# strip() – removes spaces on both sides
print(name.strip())          # Output: "JOHN DOE"

# lstrip() – removes spaces on the left only
name2 = "  hello world  "
print(len(name2.lstrip()))   # Output: 13 (right spaces still there)

# rstrip() – removes spaces on the right only
print(len(name2.rstrip()))   # Output: 11 (left spaces still there)

# Practical use – clean before comparing
user_input = "  tanishq  "
if user_input.strip() == "tanishq":
    print("Match found!")
```

2.4 – 2.5 `lower()` • `upper()` • `title()`

Case-conversion methods are crucial for **case-insensitive comparisons** and for presenting data in a consistent, readable format.

```
name = "manAV sood sinha singh"

# upper() – all characters uppercase
print(name.strip().upper())  # Output: MANAV SOOD SINHA SINGH

# lower() – all characters lowercase
print(name.strip().lower())  # Output: manav sood sinha singh

# title() – first letter of every word is capitalised
print(name.strip().title())  # Output: Manav Sood Sinha Singh

# Chaining – combine methods
email = "  USER@GMAIL.COM  "
clean_email = email.strip().lower()
print(clean_email)           # Output: user@gmail.com
```

Pro Tip — Method Chaining: Because each string method returns a new string, you can chain multiple methods together: `name.strip().lower().title()`. Python evaluates left to right.

3 Searching & Replacing

Python provides powerful built-in methods to search within strings, count occurrences, check boundaries, and replace content. These are heavily used in data parsing, validation, and text processing.

Method	Description
<code>find(sub)</code>	Returns the index of the first occurrence of sub; -1 if not found
<code>replace(old, new)</code>	Returns a new string with all occurrences of old replaced by new
<code>count(sub)</code>	Returns the number of times sub appears in the string
<code>startswith(pre)</code>	Returns True if the string begins with the given prefix
<code>endswith(suf)</code>	Returns True if the string ends with the given suffix

3.1 find()

find() searches for a substring and returns its *starting index*. If the substring is not present, it returns **-1**. This makes it perfect for conditional checks.

```
name = "manAV sood sinha singh  manav manav  "

# find() returns the index of the first match
index = name.strip().lower().find("manav")
print(index)           # Output: 0

# Using find() for conditional logic
if name.strip().lower().find("manav") == -1:
    print("not found!")
else:
    print("Found")

# Checking for @ in email
email = "tanishq@gmail.com"
print(email.find("@"))  # Output: 7
```

Python

3.2 replace()

replace(old, new) replaces every occurrence of the old substring with the new one. Since strings are immutable, this returns a *new string* — the original is unchanged.

```
Python
sentence = "I love Python. Python is great!"

# Replace all occurrences
new_sentence = sentence.replace("Python", "programming")
print(new_sentence)
# Output: I love programming. programming is great!

# Remove spaces by replacing with empty string
name = "  T a n i s h q T y a g i  "
compact = name.lower().strip().replace(" ", "")
print(compact)          # Output: tanishqtyagi
```

3.3 count()

count(sub) counts how many times a substring appears. It is case-sensitive by default — always normalise case first for reliable results.

```
Python
name = "manAV sood sinha singh  manav manav  "

# Count occurrences (case-sensitive)
print(name.count("manav"))          # Output: 2  (lowercase only)
print(name.lower().count("manav"))  # Output: 3  (all occurrences)

# Count vowels example
sentence = "Hello World"
vowel_count = sum(sentence.lower().count(v) for v in "aeiou")
print(vowel_count)                  # Output: 3
```

3.4 startswith()

startswith(prefix) returns **True** if the string begins with the specified prefix, otherwise **False**. Commonly used for validating user input or categorising data.

Python

```
name = "    T a n i s h q T y a g i    "

# startswith – check prefix after cleaning
clean = name.lower().replace(" ", "")
if clean.startswith("tan"):
    print("Yes! name starts with Tan!")
else:
    print("No, name does not start with Tan!")

# Output: Yes! name starts with Tan!

# Practical use – checking URL protocol
url = "https://example.com"
if url.startswith("https"):
    print("Secure connection!")
```

3.5 endswith()

endswith(suffix) returns **True** if the string ends with the given suffix. It is frequently used for email validation, file type detection, and URL checks.

Python

```
email_id = "tanishq@gmail.com"

# Check for valid email format
if email_id.endswith(".com") and email_id.find("@") != -1:
    print("valid email!")
# Output: valid email!

# Checking file types
filename = "notes.pdf"
if filename.endswith(".pdf"):
    print("This is a PDF file")

# Multiple suffixes
image = "photo.jpg"
if image.endswith((".jpg", ".png", ".gif")):
    print("Valid image format")
```

4 Parsing Techniques — Breaking Data Apart

Parsing means breaking a large piece of data into smaller, more manageable pieces. Python's `split()` and `join()` methods are the primary tools for converting between strings and lists.

Method	Description
<code>split(delim)</code>	Splits a string into a list of substrings using a delimiter
<code>join(iterable)</code>	Joins elements of a list into a single string using a separator

4.1 `split()`

`split(delimiter)` breaks a string into a *list* of substrings wherever the delimiter appears. If no delimiter is provided, it splits on any whitespace (spaces, tabs, newlines).

```
Python
# Split comma-separated values
fruits = "apple,banana,chiku,mango"
fruits_list = fruits.split(",")      # delimiter = ","
print(fruits_list)
# Output: ['apple', 'banana', 'chiku', 'mango']

# Split a sentence into words (default: whitespace)
sentence = "Hello World Python"
words = sentence.split()
print(words)
# Output: ['Hello', 'World', 'Python']

# Real-world: parse a CSV line
csv_row = "Tanishq,22,Mumbai,Python"
fields = csv_row.split(",")
name, age, city, lang = fields
print(f"Name: {name}, City: {city}")
```

4.2 `join()`

`join(iterable)` is the *inverse* of `split`. It takes a list of strings and concatenates them into a single string, placing the separator between each element. Note: all elements of the list must be strings.

Python

```
# Join a list of names
name = ["Tanishq", "Tyagi"]
print("$".join(name))      # Output: Tanishq$Tyagi
print(" ".join(name))     # Output: Tanishq Tyagi

# Join words back after split
words = ["Python", "is", "awesome"]
sentence = " ".join(words)
print(sentence)           # Output: Python is awesome

# Common mistake – join needs strings!
num_list = [1, 2, 3]
# print(",".join(num_list)) # ■ TypeError
print(",".join(str(n) for n in num_list)) # ■ Output: 1,2,3
```

split() and join() are inverses: `",".join("a,b,c".split(","))` gives back `'a,b,c'`. They are the foundation of CSV parsing, log file analysis, and any text data pipeline.



Quick Reference — All Methods at a Glance

Method	What it does	Example
<code>strip()</code>	Remove spaces from both ends	<code>' hi '.strip() → 'hi'</code>
<code>lstrip()</code>	Remove spaces from left end	<code>' hi '.lstrip() → 'hi '</code>
<code>rstrip()</code>	Remove spaces from right end	<code>' hi '.rstrip() → ' hi'</code>
<code>lower()</code>	Convert to all lowercase	<code>'HELLO'.lower() → 'hello'</code>
<code>upper()</code>	Convert to all uppercase	<code>'hello'.upper() → 'HELLO'</code>
<code>title()</code>	Capitalise each word	<code>'hi world'.title() → 'Hi World'</code>
<code>find(s)</code>	Index of first occurrence / -1	<code>'hello'.find('l') → 2</code>
<code>replace(o,n)</code>	Replace all occurrences	<code>'cat'.replace('c','b') → 'bat'</code>
<code>count(s)</code>	Count occurrences of substring	<code>'banana'.count('a') → 3</code>
<code>startswith(p)</code>	True if string starts with prefix	<code>'hello'.startswith('he') → True</code>

endswith(s)	True if string ends with suffix	'hello'.endswith('lo') → True
split(d)	Split into list on delimiter	'a,b'.split(',') → ['a','b']
join(lst)	Join list into string	'-'.join(['a','b']) → 'a-b'
capitalize()	Capitalise first character only	'hello world'.capitalize() → 'Hello world'
in operator	Check if substring exists	'lo' in 'hello' → True